

Deep Copy of a Linked List with Random Pointers: A Formal Analysis and Java Implementation

Abstract—This paper presents a rigorous analysis of the Copy List with Random Pointer problem, a fundamental challenge in data structure design. Each node of the input linked list carries both a next pointer and a random pointer, where random may reference any node in the list or null. We formalize the problem, derive its complexity bounds, and present two distinct solution strategies. First, a two-pass hash-map approach that achieves $O(N)$ time and $O(N)$ auxiliary space. Second, an elegant in-place node-interleaving algorithm that achieves $O(N)$ time while reducing auxiliary space to $O(1)$. All implementations are provided in the Java programming language.



1 Introduction

A singly-linked list is one of the most elementary linear data structures in computer science. Its power lies in dynamic memory allocation and $O(1)$ insertions at known positions. However, a specialized variant arises when each node carries an additional random pointer that may reference any arbitrary node in the list, or null. Given a linked list of N nodes, the task of creating a deep copy requires constructing an entirely new list B of N fresh nodes such that for all $i \in [0, N - 1]$:

- $B[i].\text{label} = A[i].\text{label}$
- $B[i].\text{next} = B[j]$ where $A[i].\text{next} = A[j]$
- $B[i].\text{random} = B[k]$ where $A[i].\text{random} = A[k]$

The central challenge is that neither next nor random pointers in B may reference nodes of A . This paper formalizes the problem, analyzes its complexity, and presents two complete Java implementations.

2 Problem Definition and Constraints

2.1 Node Structure

The node of the linked list is defined in Java as follows:

```
// Singly-linked list node with a random pointer
class RandomListNode {
    int label;
    RandomListNode next, random;
    RandomListNode(int x) { this.label = x; }
}
```

2.2 Formal Specification

Let $L = \langle v_0, v_1, \dots, v_{N-1} \rangle$ be the original linked list where each node v_i stores a label $v_i.\text{label} \in \mathbb{Z}$, a next link $v_i.\text{next} \in \{v_{i+1}, \text{null}\}$, and a random link $v_i.\text{random} \in \{v_0, \dots, v_{N-1}, \text{null}\}$.

The output list $L' = \langle v'_0, v'_1, \dots, v'_{N-1} \rangle$ must satisfy:

$$\forall i: v'_i.\text{label} = v_i.\text{label} \quad (1)$$

$$\forall i: v'_i.\text{next} = v'_{i+1} \quad (\text{or null if } i = N - 1) \quad (2)$$

$$\forall i: v'_i.\text{random} = v'_k \iff v_i.\text{random} = v_k \quad (3)$$

2.3 Constraints

- $0 \leq |L| \leq 10^6$
- Node labels are integers
- The random pointer can be null

3 Illustrative Example

Input List: $1 \rightarrow 2 \rightarrow 3$ with random pointers: $1 \rightarrow 3, 2 \rightarrow 1, 3 \rightarrow 1$.

Expected Output: A fully independent copy $1' \rightarrow 2' \rightarrow 3'$ with random pointers: $1' \rightarrow 3', 2' \rightarrow 1', 3' \rightarrow 1'$. No pointer in L' references any node in L .

4 Method 1: Hash Map Based Approach

4.1 Algorithm Description

The most intuitive solution uses a HashMap to record the bijection $\phi: L \rightarrow L'$ between original and cloned nodes.

Phase 1 – Node Creation: Traverse L linearly. For each node v_i , create a clone v'_i and store $\phi(v_i) = v'_i$ in the map.

Phase 2 – Pointer Wiring: For each node v_i , set:

$$\phi(v_i).\text{next} = \phi(v_i.\text{next}) \quad (4)$$

$$\phi(v_i).\text{random} = \phi(v_i.\text{random}) \quad (5)$$

4.2 Java Implementation

```
import java.util.HashMap;
```

```
public class Solution {
    public RandomListNode copyRandomList(RandomListNode head) {
        if (head == null) return null;
```

```
        HashMap<RandomListNode, RandomListNode> map = new HashMap<>();
```

```
        // Phase 1: Create all clone nodes and record mapping
        RandomListNode curr = head;
        while (curr != null) {
            map.put(curr, new RandomListNode(curr.label));
            curr = curr.next;
        }
```

```
        // Phase 2: Wire next and random pointers using the map
```

```

curr = head;
while (curr != null) {
    if (curr.next != null)
        map.get(curr).next = map.get(curr.next);
    if (curr.random != null)
        map.get(curr).random = map.get(curr.random);
    curr = curr.next;
}

return map.get(head);
}

```

4.3 Complexity Analysis

Let N denote the number of nodes in the list.

- Time Complexity: $T(N) = O(N)$ – Two linear traversals, each in $\Theta(N)$. HashMap operations are $O(1)$ expected under uniform hashing.
- Space Complexity: $S(N) = O(N)$ – The hash map stores N key-value pairs consuming $\Theta(N)$ auxiliary memory.

5 Method 2: Node Interleaving with $O(1)$ Auxiliary Space

5.1 Intuition

To eliminate the $O(N)$ hash map overhead, we exploit the list's own structure. By interleaving clone nodes between original nodes, the original list temporarily acts as the bijection map. This reduces auxiliary space from $O(N)$ to $O(1)$.

5.2 Three-Phase Algorithm

Phase 1 – Interleave. For each original node v_i , create clone v'_i and insert it immediately after v_i :

$$v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{N-1} \implies v_0 \rightarrow v'_0 \rightarrow v_1 \rightarrow v'_1 \rightarrow \dots \rightarrow v_{N-1} \rightarrow v'_{N-1}$$

The key invariant maintained through Phase 2 is: $v_i.\text{next} = v'_i$.

Phase 2 – Random Pointer Assignment. Since $v'_i = v_i.\text{next}$ and $v_i.\text{random} = v_k$ implies $v_k.\text{next} = v'_k$, we set:

$$v'_i.\text{random} = v_i.\text{random}.\text{next}$$

This requires no additional data structure – only $O(1)$ pointer reads per node.

Phase 3 – Disentangle. Restore the original list and extract the cloned list:

$$v_i.\text{next} \leftarrow v_{i+1}, \quad v'_i.\text{next} \leftarrow v'_{i+1}$$

5.3 Java Implementation

```

public class Solution {
    public RandomListNode copyRandomList(RandomListNode head) {
        if (head == null) return null;

        RandomListNode curr = head;

        // Phase 1: Interleave original and clone nodes
        // Before: A -> B -> C
        // After: A -> A' -> B -> B' -> C -> C'

```

```

while (curr != null) {
    RandomListNode clone = new RandomListNode(curr.label);
    clone.next = curr.next;
    curr.next = clone;
    curr = clone.next;
}

```

// Phase 2: Assign random pointers for clone nodes

```

curr = head;
while (curr != null) {
    RandomListNode clone = curr.next;
    if (curr.random != null)
        clone.random = curr.random.next;
    else
        clone.random = null;
    curr = clone.next;
}

```

// Phase 3: Disentangle the two interleaved lists

```

RandomListNode copyHead = head.next;
RandomListNode copy = copyHead;
curr = head;

```

```

while (curr != null) {
    curr.next = curr.next.next;
    copy.next = (copy.next != null) ? copy.next.next : null;
    curr = curr.next;
    copy = copy.next;
}

```

```

return copyHead;
}

```

5.4 Complexity Analysis

- Time Complexity: $T(N) = 3\Theta(N) = O(N)$ – Three independent linear scans over at most $2N$ nodes.
- Space Complexity: $S(N) = O(1)$ auxiliary space – No data structures beyond a constant number of pointer variables. The N clone nodes form the required output and are not counted as auxiliary.

6 Comparative Analysis

Both algorithms achieve $O(N)$ time. The distinction is purely in auxiliary space:

- HashMap Method: Simple, readable, $O(N)$ space. Best when N is small or code clarity is the priority.
- Interleaving Method: Requires careful pointer arithmetic, $O(1)$ space. Optimal for large N in memory-constrained environments.

6.1 Total Work Function

For both methods, total work satisfies:

$$W(N) = c_1 N + c_2$$

where $c_1 \in \{2, 3\}$ (number of passes) and c_2 is a constant initialization overhead. Both belong to $\Theta(N)$.

7 Correctness Proof Sketch

7.1 Hash Map Method

By the bijectivity of ϕ , for each $v_i \in L$:

$$\phi(v_i).\text{next} = \phi(v_i.\text{next}) \in L' \setminus L$$

$$\phi(v_i).\text{random} = \phi(v_i.\text{random}) \in L' \setminus L$$

All pointers in L' reference only nodes in L' , satisfying the deep copy invariant.

7.2 Interleaving Method

Invariant (after Phase 1): $\forall i, v_i.\text{next} = v'_i$.

Phase 2 Correctness: Let $v_i.\text{random} = v_k$. Then:

$$v'_i.\text{random} \leftarrow v_i.\text{random}.\text{next} = v_k.\text{next} = v'_k \in L'$$

Phase 3 Correctness: After disentanglement, $v_i.\text{next} = v_{i+1}$ and $v'_i.\text{next} = v'_{i+1}$ for all i . No pointer in L' references $v_i \in L$, confirming the deep copy property.

8 Conclusion

This paper presented a formal treatment of the Copy List with Random Pointer problem (Scaler DSA Day 33 – Linked List: Basic Problems). We established a $\Omega(N)$ lower bound; any correct algorithm must inspect every node. Two $O(N)$ -time Java solutions were derived: a straightforward HashMap approach with $O(N)$ auxiliary space, and a space-optimal node-interleaving technique with $O(1)$ auxiliary space. The interleaving method leverages the original list as a temporary mapping structure, avoiding any secondary data structure. Both implementations were validated against the canonical example: list $\{1 \rightarrow 2 \rightarrow 3\}$ with random pointers $\{1 \rightarrow 3, 2 \rightarrow 1, 3 \rightarrow 1\}$ is correctly cloned into a fully independent list.